Aufgabe 1: Schmucknachrichten

Teilnahme-ID: 76823

Bearbeiter/-in dieser Aufgabe:
Philipp Strieder

28. April 2025

Inhaltsverzeichnis

1 Lösungsidee 1

2 Umsetzung 2

3 Laufzeit 3

4 Beispiele 4

5 Quellcode 4

Abstract

Wir lösen Aufgabenteil a) mithilfe der Huffman-Kodierung. Dafür ordnen wir jedem Zeichen des Codierungsalphabets einen Knoten zu. Dann fassen wir immer die „seltensten“ n Knoten zu einem zusammen. So bauen wir von den Blättern zu der Wurzel einen k-ärer Huffman-Baum.
Zusätzlich wurde ein (suboptimaler) Shannon-Fano-Algorithmus implementiert, der top-down agiert und Zeichen in „gleichgewichtige“ Gruppen aufteilt.
Aufgabenteil b) lösen wir über dynamische Programmierung, indem wir jedem Huffman-Tree eine Signatur zuweisen. Ausgehend von einer Startsignatur erzeugen wir alle erreichbaren Folgesignaturen und tracken die Kosten der Kostruktion mit.

1 Lösungsidee

Teilaufgabe a)

Zunächst zählen wir die Häufigkeit jedes Quellsymbols und sortieren diese absteigend. Jedem Symbol wird ein Knoten mit einem der Häufigkeit entsprechendem Gewicht zugeordnet. Dann werden immer die n leichtesten Teilbäume zu einem neuen Baum zusammengefasst, dem die Summe ihrer Gewichte zugeordnet wird. Dieser Vorgang wiederholt sich, bis ein einzelner Baum übrig bleibt. Dieser ist der gesuchte Huffman-Tree.
Für Huffman-Bäume mit Arität $b > 2$ ist die Anzahl der Teilbäume invariant modulo $(b-1)$. Dies liegt daran, dass in jedem Schritt b Teilbäume zu einem neuen zusammengefasst werden, die Gesamtzahl der Bäume also um $b-1$ abnimmt. Da am Ende genau ein Baum übrig bleiben soll, fügen wir „dummy“-Zeichen hinzu. Diese haben Häufigkeit 0, werden also ganz unten im Baum liegen und lassen daher die „guten“ hohen Plätze im Baum für die echten Zeichen.
Aus dem Baum lässt sich eine Codetabelle rekonstruieren, indem die von jedem Knoten ausgehenden Kanten mit den Ziffern 1 bis n beschriftet werden. Der Code eines Quellcodezeichens ist dann die Hintereinanderreihung der Zeichen der Kanten von der Wurzel bis zum Knoten des Blattes.
Die so konstruierten Bäume sind immer optimal.

Der Shannon-Fano-Algorithmus baut auch einen Huffman-Baum, allerdings von der Wurzel zu den Blättern. Dafür werden erneut die Buchstaben nach ihrer Häufigkeit sortiert und dann immer zu b etwa gleichgewichtigen Gruppen eingeteilt. Auf jede dieser Gruppen wird dieses Verfahren dann rekursiv erneut angewandt, bis

Aufgabenteil b)

Anders als bei Aufgabenteil a) bauen wir die Bäume jetzt nicht mehr von den Blättern zur Wurzel, sondern von der Wurzel zu den Blättern auf. Wir orientieren uns dabei an dem Paper *„A Dynamic Programming Algorithm for Constructing Optimal Prefix-Free Codes with Unequal Letter Costs“* von Golin und Rote (1998), in welchem ein Algorithmus vorgestellt wird, der das Problem in $O(n^{C+2})$ Zeit löst, wenn die Buchstabenkosten ganzzahlig und durch eine maximale Kostenhöhe C beschränkt sind.

Die zentrale Idee besteht darin, anstelle der exakten Baumstruktur nur eine sogenannte *Signatur* zu betrachten. Eine solche Signatur

$$\mathrm{sig}_i(T) \;=\; (m,\, l_1,\, \ldots,\, l_C)$$

beschreibt den aktuellen Zustand eines (teilweise konstruierten) Huffman-Baums:

- m ist die Anzahl der Blätter bis einschließlich Ebene i,
- l_j ist die Anzahl der Knoten auf Ebene $i + j$.

Durch sogenannte *Expansionen* lassen sich neue Signaturen aus bestehenden erzeugen: Dabei werden $0 \leq q \leq l_1$ der Knoten auf Ebene $i + 1$ zu inneren Knoten gemacht, wodurch neue Kinder auf tieferen Ebenen entstehen (je nach Buchstabenkosten c_j). Der Übergang von einer Signatur zur nächsten wird rein algebraisch über eine Vektoraddition modelliert.

Für jede Signatur berechnen wir einen Kostenwert, der sich aus zwei Teilen zusammensetzt:

1. den bisherigen Kosten der Vorgängersignatur,
2. sowie dem zusätzlichen Beitrag, der entsteht, weil die restlichen (noch nicht untergebrachten) Zeichen mindestens eine Ebene tiefer kodiert werden müssen. Dieser Zusatzkostenbeitrag ist genau die Summe der $n - m$ kleinsten Frequenzen, da sich seltene Zeichen am Ende einen Level tiefer „leisten“ können.

Sobald eine Signatur der Form

$$(n, 0, 0, \ldots, 0)$$

erreicht wird – also alle n Zeichen als Blätter auf möglichst flachen Ebenen untergebracht sind – haben wir einen optimalen Codebaum gefunden. Dieser entspricht einem kürzesten Pfad im sogenannten *Signaturgraphen*, in dem jeder Knoten eine Signatur repräsentiert und jede (gerichtete) Kante eine zulässige Expansion.

Damit unser dynamischer Programmier-Ansatz korrekt funktioniert, müssen wir jede neue Signatur erst dann verarbeiten, wenn alle Signaturen, von denen sie per Expansion abhängt, schon bearbeitet sind. Dazu führen wir eine feste Reihenfolge auf den Signaturen ein, nämlich die lexikographische Ordnung ihrer *partiellen Summen*

$$\big(m + l_1 + \cdots + l_C,\; m + l_1 + \cdots + l_{C-1},\; \ldots,\; m + l_1,\; m\big).$$

Jede Expansion fügt $(r - 1)\, q$ neue Blätter hinzu (und kann bei Bedarf per Reduction wieder auf n gekürzt werden), sodass diese Folge von Summen immer „größer“ wird. Damit entstehen keine Zyklen, und wir garantieren, dass alle Vorgänger-Signaturen vor einer neuen Signatur schon bearbeitet sind. Dies ist im Wesentlichen eine topologische Sortierung des zugrundeliegenden, gerichteten azyklischen Graphen.

2 Umsetzung

Teilaufgabe a)

Die Implementierung erfolgt in modernem Python (Version 3.8+), wobei alle Zeichen als Unicode-Zeichen kodiert werden. Zunächst wird mithilfe eines Dictionaries die Häufigkeit jedes Zeichens im Eingabetext in $O(n)$ Zeit gezählt. Diese Paare (Zeichen, Frequenz) werden in zwei parallele Listen überführt, die nach ihren Frequenzen absteigend sortiert werden ($O(d \log d)$ mit d unterschiedlichen Zeichen).

Die b-äre Huffman-Kodierung wird durch folgende Schritte realisiert:

1. Hinzufügen der dummies: Solange sich die Knotenzahl nicht der Form $1 \bmod (b-1)$ entspricht, werden Dummy-Knoten mit Gewicht 0 angehängt. Dies garantiert, dass sich der Algorithmus korrekt terminiert.

2. Baumaufbau: In einer Schleife werden stets die b kleinsten Teillisten entfernt und zu einem neuen Knoten mit kombinierter Frequenz zusammengeführt. Zur Effizienz kann sowohl eine sortierte Liste als auch ein `heapq`-basiertes Minimum-Heap genutzt werden. Unsere Listen-Variante benötigt $O(d \log d/(b-1))$, ein Min-Heap würde ebenfalls $O(d \log d)$ erreichen.

3. Codebuch-Extraktion: Eine einfache rekursive Funktion durchläuft den verschachtelten Baum (repräsentiert als verschachtelte Listen) und schreibt dabei den Codepfad (Index der Kindkante) in ein Wörterbuch. Dies geschieht in $O(d)$, da jeder Baumknoten genau einmal besucht wird.

Die Shannon-Fano-Implementierung teilt das Zeichenalphabet top-down in b möglichst gleichgewichtige Gruppen, indem sukzessiv Teilsummen gebildet werden. Diese rekursive Zerlegung benötigt im Worst-Case $O(d \log d)$, ist aber in der Praxis meist langsamer als Huffman-Kodierung und suboptimal.

Teilaufgabe b)

Der dynamische Algorithmus aus Golin und Rote (1998) modelliert einen teilweisen Huffman-Baum nicht als konkrete Struktur, sondern als eine *Signatur*-Vektor der Länge $C+1$, wobei C die maximale Buchstabenkostenhöhe ist. Diese Signatur enthält:

- m: Anzahl der bereits abgeschlossenen Blätter bis zur aktuellen Ebene.
- $l_1, \ldots, l_C$: Anzahl der offenen Knoten in den darauf folgenden Ebenen.

Statt alle theoretisch möglichen Signaturen zu generieren, verwenden wir eine Prioritätswarteschlange (Min-Heap), in der stets nur tatsächlich erreichbare Signaturen in lexikographischer Topo-Reihenfolge der partiellen Summen bearbeitet werden. Dies erhöht zwar die asymptotische Laufzeit, ist allerdings in der Praxis schneller, da wir deutlich weniger Tupel betrachten müssen und keine aufwendige Tupelgenerierung betreiben müssen (Eine von mir geschriebene Version des Programms, die Tupel vorgenerierte brauchte selbst nach starken Optimierungen für schmuck5 ca. $45s$ statt $0.7s$; den Großteil dieser Zeit verbrachte sie mit der Tupelgenerierung). Jeder Heap-Eintrag enthält die Signatur und die bisher minimalen Kosten. Die Übergänge erfolgen in drei Schritten:

1. `shift_signature`: Verschiebe die Ebenen um eins nach vorne, sodass neue Knoten eine Ebene tiefer beginnen.

2. `add_to_signature`: Für jede zulässige Anzahl $0 \leq q \leq l_1$ offener Knoten wird eine neue Signatur erzeugt, indem man q Knoten auflöst und ihre Kinder gemäß Buchstabenkosten hinzufügt.

3. `reduce_signature`: Falls dabei mehr als n Blätter entstehen, werden die tieferen abgeschnitten, da sie teurer sind.

Für jede neue Signatur werden die Kosten aus der Vorgänger-Signatur und einem Restkostenanteil (Summe der Frequenzen der noch nicht platzierten Zeichen) berechnet. Der Algorithmus terminiert, sobald eine Signatur mit insgesamt n Blättern in Ebene 0 erreicht ist. Die Rückverfolgung des optimalen Pfads erfolgt über das Dictionary `path`, das zu jeder Signatur den gewählten Expansionsschritt speichert. Über eine abschließende Rekonstruktionsfunktion wird so das optimale Codebuch generiert.

3 Laufzeit

Sei n die Länge des Eingabetextes, d die Anzahl unterschiedlicher Zeichen und b die Arität.

- Häufigkeiten zählen: $O(n)$
- Sortieren: $O(d \log d)$
- Dummy-Erzeugung: $O(b)$
- Huffman-Baum: $O(d \log d/(b-1))$

- Umwandlung in Codebook und Längenberechnung: $O(k)$

Insgesamt ergibt sich für Teil a): $O(n + d \log d)$. Dies könnte man durch Verwendung eines min-heaps auf $O(\frac{d \log d}{b})$ bringen, allerdings ist das Programm bereits schnell genug um alle Beispiele zu lösen. Da das Programm nur Unicode-Zeichen betrachten soll und dieser aktuell nur 154.998 Zeichen enthält, ist diese Laufzeit nach oben begrenzt und wir können eigentlich von einer worst-case-Laufzeit $O(n + 154.998 log(154.998) = O(n + 1) = O(n)$ reden.

Für Teil b): Der dynamische Programmier-Ansatz untersucht nur tatsächlich erreichbare Signaturen. Im schlimmsten Fall können bis zu $O(n^{C+1})$ verschiedene Signaturen entstehen, und jede Expansion (bis zu n Schritte pro Signatur) kann $O(\log(N_{states}))$ Zeit im Heap erfordern. Damit liegt die Laufzeit im Worst-Case bei

$$O\big(n^{C+1} \times n \times \log(n^{C+1})\big) \;=\; O\big(n^{C+2}\,(C+1)\log n\big) \approx O(n^{C+2}\log n)\,.$$

Der praktische Aufwand ist jedoch deutlich geringer, da die Anzahl der tatsächlich besuchten Signaturen und Expansionsschritte durch die Frequenzverteilung beschränkt wird. Würden wir die Tupel vorgenerieren, fälle der $\log n$-Teil weg und wir hätten eine asymptotische Laufzeit von $O(n^{C+2})$.
Es wurde bisher weder eine Lösung in polynomieller Zeit noch ein Beweis von NP-Vollständigkeit für dieses Problem gefunden (Zumindest behaupten das 1998 Golin und Rote und 2025 ChatGPT).

4 Beispiele

Der Lesbarkeit zuliebe werden hier nur die erreichten Längen angegeben. Die vollen Codetabellen sollten sich in externen Text-Dateien befinden. Zusätzlich zu den gegebenen Dateien habe ich noch einige weitere Beispielseingaben entworfen. Die beiden Bibeltexte sollen vor allem demonstrieren, dass die Aezahl der Zeichen im Text die Laufzeit nicht großartig beeinflusst. Die Datei deutscher Morsecode enthält Buchstaben der deutschen Sprache entsprechend ihrer Häufigkeit.
Teilaufgabe a) (Codierlänge in mm):

Datei	Huffman	Shannon-Fano
schmuck0:	113	114
schmuck00:	372	389
schmuck01:	1150	1355

Teilaufgabe b) (Codierlänge in mm):

schmuck1:	191
schmuck2:	135
schmuck3:	279
schmuck4:	137
schmuck5:	3162
schmuck6:	234
schmuck7:	134559
schmuck8:	3287
schmuck9:	???
schmuck10:	5386
schmuck_bibel_base2:	19468020
schmuck_bibel_base9:	13986037
schmuck_deutscher_morsecode:	5466

Bei schmuck9 hatte mein Arbeitsspeicher leider keine Lust mehr. Schade. Vielleicht können Sie das Programm auf ihrem PC, der sicherlich besser ist als mein Laptop, mal laufen lassen und überprüfen, ob das an meiner Hardware oder an meinem suboptimalen Code lag.

5 Quellcode

Teilaufgabe a) – n-äre Huffman-Kodierung

```
# Haeufigkeiten zaehlen
for c in txt:
```

```python
    if c in chars:
        freq[chars.index(c)] += 1
    else:
        chars.append(c)
        freq.append(1)

# Dummy-Knoten hinzufuegen, falls noetig
while len(freq) % (base - 1) != 1 and base != 2:
    chars.append('dummy' + str(dc))
    freq.append(0)
    dc += 1

# Huffman-Baum aufbauen
while len(nodes) > base:
    newnode = []
    f = 0
    for i in range(base):
        newnode.append(nodes.pop())
        f += nodef.pop()
    k = 0
    while k < len(nodef) and f < nodef[k]:
        k += 1
    nodef.insert(k, f)
    nodes.insert(k, newnode)

# Codebook generieren
def trtocb(tree):
    if len(tree) == 1:
        return ([tree[0]], [[]])
    else:
        letters, codes = [], []
        for i in range(base):
            l, c = trtocb(tree[i])
            for j in range(len(l)):
                letters.append(l[j])
                codes.append([i] + c[j])
        return (letters, codes)
```

Listing 1: Huffman-Kodierung

Teilaufgabe a) – Shannon-Fano-Kodierung

```python
def shannonfano(chars, freq):
    if len(chars) == 1:
        return [chars[0]]
    cb = []
    split = [0]
    s = sum(freq)
    rs = 0
    k = 1
    for i in range(len(freq)):
        rs += freq[i]
        if rs >= k * s / base:
            k += 1
            c = abs(rs - k * s / base) < abs(rs - freq[i] - k * s / base)
            split.append(i + 1 if c else i)
    if split[-1] != len(chars):
        split.append(len(chars))
    for i in range(len(split) - 1):
        sub_chars = chars[split[i]:split[i+1]]
        sub_freq = freq[split[i]:split[i+1]]
        sub_codes = shannonfano(sub_chars, sub_freq)
        cb += [str(i) + code for code in sub_codes]
    return cb
```

Listing 2: Shannon-Fano Algorithmus (rekursiv)

Teilaufgabe b) – Prefix-Code mit variablen Kosten

```python
def shift_signature(sig, C):
    return [([sig[0]] + C * [0])[i] + sig[i+1] for i in range(C)] + [0]

def add_to_signature(sig, q, dv, C):
    return [sig[i] + q * dv[i] for i in range(C + 1)]

def reduce_signature(sig, n, C):
    cs = 0
    new_sig = []
    i = 0
    while cs < n:
        cs += sig[i]
        i += 1
    for j in range(i - 1):
        new_sig.append(sig[j])
    new_sig.append(sig[i - 1] - cs + n)
    for j in range(i, C + 1):
        new_sig.append(0)
    return new_sig
```

Listing 3: Kernalgorithmus zur Generierung optimaler Signaturen

```python
# Initialzustand
initial_state = (0,) + d
if sum(initial_state) > n:
    initial_state = reduce_signature(initial_state, n, C)

optimal_costs = {initial_state: 0}
paths = {initial_state: []}
heapq.heappush(queue, (priority(initial_state, C), initial_state))

while queue:
    cp, sig = heapq.heappop(queue)
    m = sig[0]
    new_cost = optimal_costs[tuple(sig)] + sum(p[m:])
    shifted = shift_signature(sig, C)

    for q in range(sig[1] + 1):
        new_sig = add_to_signature(shifted, q, dv, C)
        if sum(new_sig) > n:
            new_sig = reduce_signature(new_sig, n, C)
        if new_sig not in optimal_costs or new_cost < optimal_costs[tuple(new_sig)]:
            optimal_costs[tuple(new_sig)] = new_cost
            paths[tuple(new_sig)] = paths[tuple(sig)] + [q]
            heapq.heappush(queue, (priority(new_sig, C), new_sig))
```

Listing 4: Heap-basierter Traversierungsalgorithmus

```python
def path_to_cb(path, letters, c):
    r = len(c)
    C = max(c)
    nodes = [[] for _ in range(C)]
    cl = -1
    for i in range(r):
        nodes[c[i] + cl].append([i])
    for step in path:
        cl += 1
        nodes.append([])
        for _ in range(step):
            cp = nodes[cl].pop()
            for j in range(r):
                nodes[cl + c[j]].append(cp + [j])
    cb = {}
    j = 0
    for char in letters:
        while not nodes[j]:
            j += 1
        cb[char] = nodes[j].pop()
    return cb
```

Listing 5: Rekonstruktion des Codebooks aus Pfad